

Prompt Iteration Log: FL-02 Prompting Fundamentals

Target Task: Generating a production-ready FastAPI router module with database integration, Pydantic schemas, dependency injection, and proper error handling.

1. Prompt Iterations & Output Log

Version 0: Naive Prompt (Baseline)

- **Technique Applied:** Baseline (Unengineered / Naive)
- **Prompt:**
Plaintext

Write a FastAPI router for user management with database stuff and endpoints.
- **LLM Output Summary:** Generates a basic 10-line router returning hardcoded JSON dictionaries. Uses basic synchronous endpoints, no Pydantic validation schemas, no database connections, and `user: dict` for request bodies.
- **Observational Note:** The raw baseline produces toy code. It lacks real-world software practices, missing database session handling, status codes, and input validation.

Version 1: Role Assignment

- **Technique Applied:** Persona / Role Assignment
- **Prompt:**
Plaintext

You are a Senior Python Backend Architect specializing in high-performance FastAPI microservices and clean architecture.

Write a FastAPI router for user management with database stuff and endpoints.
- **LLM Output Summary:** Introduces standard FastAPI conventions: includes router prefixing (`prefix="/users"`), API tagging, explicit status codes (`status_code=201`), placeholder dependency injection (`db: Session = Depends()`), and structured `HTTPException` raises.
- **Observational Note:** Simply assigning the senior role instantly elevates code hygiene and API structure, even without changing the core task instructions.

Version 2: Context and Motivation

- **Technique Applied:** Context and Motivation
- **Prompt:**

Plaintext

You are a Senior Python Backend Architect specializing in high-performance FastAPI microservices and clean architecture.

We are building an asynchronous, production-grade SaaS API backend using FastAPI, SQLAlchemy 2.0 (async), and PostgreSQL. The endpoint router must handle user CRUD operations safely, ensuring strict type safety and relational database persistence without leaking database exceptions directly to the user.

Write a FastAPI router for user management with database stuff and endpoints.

- **LLM Output Summary:** Switches the implementation paradigm from synchronous to asynchronous Python (`AsyncSession`, `await db.execute()`). Wraps raw database queries in `try/except` blocks to prevent database stack traces from leaking to the client.
- **Observational Note:** Providing technical context forces the model to choose modern, production-appropriate libraries and error handling paradigms.

Version 3: Output Structure Constraints

- **Technique Applied:** Explicit Output Formatting & Constraints
- **Prompt:**

Plaintext

You are a Senior Python Backend Architect specializing in high-performance FastAPI microservices and clean architecture.

We are building an asynchronous, production-grade SaaS API backend using FastAPI, SQLAlchemy 2.0 (async), and PostgreSQL. The endpoint router must handle user CRUD operations safely, ensuring strict type safety and relational database persistence without leaking database exceptions directly to the user.

Please structure your output into exactly three clear sections:

1. `### Pydantic Schemas``: Standard request/response schemas (`UserCreate`, `UserResponse`) using Pydantic v2 `ConfigDict``.
2. `### FastAPI Router Implementation``: Complete async endpoint implementation (POST, GET by ID) with full type annotations.
3. `### Security & Exception Notes``: Bullet points detailing edge cases handled (e.g., duplicate entries, 404s).

Do not include conversational filler or setup explanations outside these sections.

- **LLM Output Summary:** Organizes the output strictly under the three requested markdown headings. Separates Pydantic v2 request/response validation models from the router code and adds structured notes on exception handling. Eliminates conversational intros.
- **Observational Note:** Structural constraints prevent the model from lumping schemas and endpoints into a single messy block, making the output modular and readable.

Version 4: Few-Shot Examples

- **Technique Applied:** Few-Shot Exemplars

- **Prompt:**

Plaintext

You are a Senior Python Backend Architect specializing in high-performance FastAPI microservices and clean architecture.

We are building an asynchronous, production-grade SaaS API backend using FastAPI, SQLAlchemy 2.0 (async), and PostgreSQL.

Structure your response into:

1. `### Pydantic Schemas`
2. `### FastAPI Router Implementation`
3. `### Security & Exception Notes`

Follow this exact schema and routing pattern example:

```
<example>
class ItemCreate(BaseModel):
    title: str = Field(..., min_length=1, max_length=100)

class ItemResponse(BaseModel):
    id: int
    title: str
    model_config = ConfigDict(from_attributes=True)

@router.post("/", response_model=ItemResponse,
status_code=status.HTTP_201_CREATED)
async def create_item(payload: ItemCreate, db: AsyncSession =
Depends(get_db)) -> ItemResponse:
    # CRUD pattern here...
</example>
```

Now write the full implementation for User management (Create User and Get User by ID).

- **LLM Output Summary:** Mirrors the example pattern accurately: applies string length bounds (`Field(..., min_length=2)`), explicit return type annotations (`-> UserResponse`), transactional rollbacks (`await db.rollback()`), and specific HTTP status codes (`409 Conflict`).
- **Observational Note:** Few-shot examples anchor the model's syntax style, ensuring precise validation rules and error-handling idioms without needing lengthy verbal explanations.

Version 5: Step Decomposition

- **Technique Applied:** Step Decomposition / Chain-of-Thought
- **Prompt:**

Plaintext

You are a Senior Python Backend Architect specializing in high-performance FastAPI microservices and clean architecture.

We are building an asynchronous, production-grade SaaS API backend using FastAPI, SQLAlchemy 2.0 (async), and PostgreSQL.

Task: Create a production-ready User Management Router (POST /users, GET /users/{id}).

Follow these steps sequentially before writing code:
Step 1: Analyze database constraints and define required Pydantic v2 schemas with validation bounds.
Step 2: Plan exception handling strategy (handling DB integrity errors, non-existent record queries, and session management).
Step 3: Implement the async router logic using strict type annotations and explicit status codes.

Structure your response into:

- 1. ````### Step-by-Step Design Strategy``
- 2. ````### Pydantic Schemas``
- 3. ````### FastAPI Router Implementation``
- 4. ````### Security & Exception Notes``

- **LLM Output Summary:** Generates a dedicated strategy breakdown prior to code generation. Evaluates race conditions, database uniqueness constraints, and session safety first, resulting in fully robust, production-grade router and schema definitions.
- **Observational Note:** Decomposition forces the LLM to "think" before writing, catching subtle bugs like unhandled session rollbacks and incomplete field validations before generating final code.

2. Cross-Model Comparison

Metric	Claude 3.5 Sonnet	ChatGPT (GPT-4o)
Tone	Concise, direct, and architectural. Zero conversational noise.	Detailed and instructional; inclines toward verbose explanations.
Accuracy	Flawless modern syntax (Pydantic v2 ConfigDict and SQLAlchemy 2.0 select()).	Mostly accurate, but tends to default to older ORM patterns unless strictly guided.
Structure	Strict adherence to requested markdown section headers.	Follows headers well, but adds introductory conversational intros/outros.
Failure Points	Assumes external imports/models exist without stubbing them.	Misses explicit transaction rollbacks (db.rollback()) on initial error catches if not directed.

3. Universal Reusable Prompt Template

Plaintext

You are an expert [ROLE/PROFESSION] specializing in [SPECIFIC DOMAIN/TECHNOLOGY].

Context & Goal:
We are building [PROJECT/SYSTEM DESCRIPTION]. The goal is to generate [DELIVERABLE TYPE] that meets production standards, following best practices for [KEY REQUIREMENT 1, KEY REQUIREMENT 2].

Instructions & Steps:

- 1. Step 1: Analyze [INPUT DATA / REQUIREMENTS] and outline key edge cases and design considerations.
- 2. Step 2: Formulate [CORE COMPONENT 1] with complete types, validation rules, and error handling.
- 3. Step 3: Implement [CORE COMPONENT 2] following clean architecture guidelines.

Formatting Requirements:

Structure your response into the following exact sections:

- `### Step 1: Architecture & Design Strategy`
- `### Step 2: Component Definition`
- `### Step 3: Core Implementation`
- `### Step 4: Edge Cases & Security Considerations`

Example Pattern:

<example>

[INSERT SAMPLE DESIRED OUTPUT PATTERN HERE]

</example>

Constraints:

- Do not use deprecated syntax or libraries.
- Avoid generic conversational introductions or conclusions outside the requested sections.